Data Analytics in Python

Module 1 Assessment - Python Foundations Question Bank

Covers Classes 0-6: Data Types, Strings & Regex, Lists/Tuples/Comprehensions, Dictionaries & Sets, Control Flow & Loops, Functions/Lambdas/Modules

INSTRUCTIONS

- These Assessment questions cover Classes C0–C6 only (Python foundations through functions/modules). No pandas, numpy, or external libraries permitted.
- Part A (Q1–Q35): Five mini-projects, 7 questions each, 2 marks per question = 70 marks.
- Part B (Q36–Q45): Ten standalone questions, 3 marks each = 30 marks.
- Efficiency and idiomatic Python are graded, not just correctness. Prefer .get() over direct key access + try/except where appropriate; prefer comprehensions over manual loops where they improve readability; prefer set operators over manual membership loops.
- Each mini-project's dataset is provided in a starter code cell - do not modify the dataset.
- Partial credit is awarded for correct logic even with minor bugs. Show your reasoning in comments.
- Where a question asks you to compare two approaches (e.g. .get() vs. dict[key], loop vs. comprehension), you must write both and add a one-line comment explaining which you would use in production code and why. This is graded.
- Write clean, readable code: meaningful variable names, no single-letter variables except loop counters, and handle edge cases (empty input, missing keys, invalid types) explicitly.

PART A - FIVE MINI-PROJECTS

Project 1 - Clinic Patient Triage Log

Context: queue triage

Difficulty: Low-Medium | Dataset: list of 8 patient dicts

```
patients = [
    {"id":"P01","name":"Ahmad Raza","age":34,"temp":38.9,"dept":"General"},
    {"id":"P02","name":"Sara Khan","age":28,"temp":36.7,"dept":"General"},
    {"id":"P03","name":"Bilal Ahmed","age":51,"temp":39.4,"dept":"ICU"},
    {"id":"P04","name":"Zara Malik","age":45,"temp":38.2,"dept":"General"},
    {"id":"P05","name":"Hassan Ali","age":67,"temp":40.1,"dept":"ICU"},
    {"id":"P06","name":"Nida Butt","age":39,"temp":36.9,"dept":"General"},
    {"id":"P07","name":"Usman Sheikh","age":29,"temp":37.5,"dept":"General"},
    {"id":"P08","name":"Ayesha Nawaz","age":31,"temp":38.6,"dept":"ICU"},
]
```

Q1. Write a function `avg_temp(patients)` that returns the average temperature across all patients (manual sum, no libraries).

Q2. Write `fever_patients(patients, threshold=37.5)` that returns a list of names with temperature above the threshold, using a list comprehension.

Q3. Write `by_department(patients)` that returns a dict mapping each department to a list of patient names in it, built with a plain for loop.

Q4. Write `oldest_patient(patients)` using `max()` with a `key=` lambda - no manual loop allowed.
Hint: max(patients, key=lambda p: p['age'])

Q5. Given a lookup dict `vitals_extra = {'P01': 'stable', 'P05': 'critical'}`, write code that prints each patient's status using `.get(pid, 'unknown')` instead of a `KeyError`-prone direct lookup. Explain in a comment why `.get()` is safer here.

Q6. Write `triage_order(patients)` that returns patient IDs sorted by temperature descending using `sorted()` with `key=` - most urgent first.

Q7. Write `department_summary(patients)` returning a dict `{dept: {'count': n, 'avg_temp': x}}` for each department using nested logic (loop or dict comprehension, your choice).

Project 2 - Mini E-Commerce Order Book

Context: small online store

Difficulty: Medium | Dataset: list of 6 order dicts, each with a nested items list

```
orders = [
    {"order_id":"O1001","customer":"Ahmad","items":[("Mouse",2,1200),("Keyboard",1,2500)]},
    {"order_id":"O1002","customer":"Sara","items":[("Monitor",1,32000)]},
    {"order_id":"O1003","customer":"Bilal","items":[("USB Hub",3,1800),("Mousepad",2,450)]},
    {"order_id":"O1004","customer":"Ahmad","items":[("Webcam",1,4500),("Headset",1,6800)]},
    {"order_id":"O1005","customer":"Zara","items":[("Laptop",1,85000)]},
    {"order_id":"O1006","customer":"Sara","items":[("Keyboard",2,2500),("Mouse",1,1200)]},
]
# each item tuple = (product_name, quantity, unit_price)
```

Q1. Write `order_total(order)` that computes the total value of one order by summing `qty * price` across its items tuple list.

Q2. Write `all_order_totals(orders)` that returns a dict `{order_id: total}` using a dict comprehension calling `order_total()`.

Q3. Write `customer_spend(orders)` that returns a dict mapping each customer name to their total spend across all their orders (a customer can have multiple orders - accumulate).

Q4. Find the order with the highest total value using `max()` with `key=` and the function from Q1 - no manual loop.

Q5. Write `most_purchased_product(orders)` that returns the product name bought in the highest total quantity across all orders (use a frequency dict, `.get(key, 0) + qty` pattern to accumulate).

Hint: running_totals[name] = running_totals.get(name, 0) + qty

Q6. Using a set, find `unique_products` - the set of every distinct product name appearing across all orders. Do this without a manual `if x not in seen` check.

Hint: Build a list of names then wrap it in set(...)

Q7. Write `generate_receipt(order)` that returns a formatted multi-line string receipt for one order (item, qty, unit price, line total, and grand total) using f-strings with column alignment.

Project 3 - Semester Exam Result Sheet

Context: university exam office

Difficulty: Medium | Dataset: dict of dicts - student ID -> subject scores

```
results = {
    "2023-CS-01": {"Math": 78, "Physics": 65, "Programming": 91},
    "2023-CS-02": {"Math": 88, "Physics": 82, "Programming": 95},
    "2023-CS-03": {"Math": 45, "Physics": 58, "Programming": 62},
    "2023-CS-04": {"Math": 92, "Physics": 89, "Programming": 84},
    "2023-CS-05": {"Math": 55, "Physics": 48, "Programming": 60},
    "2023-CS-06": {"Math": 70, "Physics": 73, "Programming": 68},
}
```

Q1. Write `student_average(scores)` that takes one student's score dict and returns the average of its values (use `.values()`, not manual key lookups).

Q2. Write `class_averages(results)` that returns `{student_id: average}` for every student using a dict comprehension calling `student_average()`.

Q3. Write `assign_grade(avg)` returning 'A' (>=85), 'B' (70-84), 'C' (50-69), 'F' (<50). Then build `grade_report` mapping each student to their grade.

Q4. Write `subject_average(results, subject)` that computes the class average for one named subject (e.g. `'Math'`) by pulling that key from every student's dict - use `.get(subject, 0)` defensively and explain in a comment what happens if a student is missing that subject.

Q5. Find the top-scoring student in `'Programming'` using `max()` with `key=` on a lambda that looks into the nested dict - no manual loop.

Hint: max(results, key=lambda sid: results[sid]['Programming'])

Q6. Write `at_risk_students(results, threshold=50)` returning a list of student IDs where their average is below the threshold, using a list comprehension.

Q7. Build `grade_distribution`: a dict counting how many students got each letter grade (A/B/C/F) using the grades from Q3, built with a dict comprehension over a set of unique grades.

Project 4 - DNA Sequence Quality Batch

Context: genomics lab QC

Difficulty: Medium-High | Dataset: list of raw DNA sequence strings (some invalid)

```
sequences = [
    "ATCGATCGATCG",
    "atcgGGCCatcg",
    "ATCGXATCG",       # contains invalid character X
    "GGGGCCCCATAT",
    "",                # empty sequence
    "ATCG-ATCG",       # contains a hyphen
    "TTTTAAAACCCC",
    "ATCGATCGATCGATCGATCG",
]
```

Q1. Write `is_valid_dna(seq)` that returns `True` only if the sequence (after uppercasing) contains ONLY A, T, C, G characters and is non-empty. Use `re.fullmatch()`.

Q2. Write `gc_content(seq)` returning GC percentage, rounded to 1 decimal place. Return `0.0` for invalid or empty sequences instead of crashing.

Q3. Using a list comprehension with a condition, build `valid_sequences` containing only the sequences that pass `is_valid_dna()`.

Q4. Build `gc_report`: a dict mapping each valid sequence (as key) to its GC content (as value), using a dict comprehension over `valid_sequences`.

Q5. Write `longest_sequence(valid_sequences)` using `max()` with `key=len` - no manual loop.

Q6. Using `re.findall()`, write `count_motif(seq, motif='ATCG')` that counts non-overlapping occurrences of a motif in one sequence.

Q7. Write a summary function `batch_report(sequences)` that returns a dict with keys: `total`, `valid_count`, `invalid_count`, `avg_gc` (average GC of valid sequences only, 0.0 if none valid).

Project 5 - Departmental Library Catalog

Context: university library system

Difficulty: Medium-High | Dataset: dict of dicts - ISBN -> book info, tags stored as a set

```
catalog = {
    "ISBN-001": {"title":"Intro to Algorithms","copies":3,"tags":{"CS","Math","Core"}},
    "ISBN-002": {"title":"Python for Data Analysis","copies":5,"tags":{"CS","Data","Elective"}},
    "ISBN-003": {"title":"Genomics Primer","copies":2,"tags":{"Bio","Data","Core"}},
    "ISBN-004": {"title":"Linear Algebra Basics","copies":0,"tags":{"Math","Core"}},
    "ISBN-005": {"title":"Statistics for Scientists","copies":4,"tags":{"Math","Data","Elective"}},
    "ISBN-006": {"title":"Machine Learning Intro","copies":1,"tags":{"CS","Data","Bio"}},
}
```

Q1. Write `out_of_stock(catalog)` returning a list of book titles where `copies == 0`, using a list comprehension.

Q2. Write `books_with_tag(catalog, tag)` returning a list of titles that contain the given tag in their tags set - use the `in` operator on a set (not a manual character search).

Q3. Build `all_tags`: the union of every book's tags set across the whole catalog, computed WITHOUT a manual loop that adds one tag at a time (use `set.union(*...)` or equivalent).

Hint: set().union((book['tags'] for book in catalog.values()))*

Q4. Write `cs_and_data_books(catalog)` returning titles of books tagged with BOTH `'CS'` AND `'Data'`, using set intersection (`&`) on each book's tags.

Q5. Write `total_copies(catalog)` that sums `copies` across the whole catalog using `sum()` with a generator expression - no manual accumulator loop.

Hint: sum(b['copies'] for b in catalog.values())

Q6. Write `restock_needed(catalog, threshold=2)` returning `{title: copies}` for books at or below the threshold, using a dict comprehension.

Q7. Write `tag_frequency(catalog)` returning a dict mapping each unique tag to how many books carry it - build it using `all_tags` from Q3 and `.count()`-style logic via a comprehension over `catalog.values()`.

PART B - STANDALONE QUESTIONS

10 Questions × 3 marks = 30 marks | Concepts span C0–C6

Q36. Efficiency check - Two candidate implementations are shown below for looking up a patient's age from a dict, with a default of ``'Unknown'` if missing:
(a) `if pid in d: age = d[pid] else: age = 'Unknown'`
(b) `age = d.get(pid, 'Unknown')`
Explain in 2-3 sentences which is preferred in production code and why (readability, performance, and risk of `KeyError` if written incorrectly).

Hint: No code required - written answer.

Q37. String cleaning - Given `raw = '  Ahmad   RAZA  '`, write a single chained expression (no intermediate variables, no loops) that produces ``'Ahmad Raza'` - stripped, single-spaced, and title-cased.

Q38. List vs comprehension - Given `nums = [4, 15, 8, 23, 16, 42, 7]`, write ONE list comprehension that returns only the even numbers, doubled. Do not use a `for` loop with `.append()`.

Q39. Tuple unpacking - Given `record = ('P004', 'Zara Malik', 45, 38.2)`, unpack it into four named variables in a single line, then print a formatted sentence using all four.

Q40. Set membership efficiency - You need to repeatedly check whether a patient ID exists in a large collection of 10,000 IDs. Explain in 1-2 sentences why storing the IDs in a `set` is more efficient for membership testing than storing them in a `list`.

Hint: Written answer - mention average-case time complexity (O(1) vs O(n)).

Q41. Control flow - Write a function `classify_bp(systolic)` using `if/elif/else` that returns ``'Low'` (<90), ``'Normal'` (90-120), ``'Elevated'` (121-139), or ``'High'` (>=140).

Q42. Loop with early exit - Given a list of temperatures `[36.5, 36.8, 37.1, 39.8, 37.0]`, write a `for` loop that stops (using `break`) as soon as it finds the first temperature above 39.0, and prints its index and value.

Q43. Lambda + sorted - Given `patients = [('Ahmad', 34), ('Sara', 28), ('Bilal', 51)]` (name, age tuples), sort this list by age descending using `sorted()` with a `lambda` as the key - in one line.

Q44. Function with default + type hint - Write a function `bmi(weight: float, height: float, unit: str = 'metric') -> float` that computes BMI (`weight / height**2` for metric in kg/m²). Include a docstring with one line describing what it returns.

Q45. Module usage - Using the `random` module, write code that sets a seed of `42`, then generates a list of 5 random integers between 1 and 100 using a single list comprehension (not five separate calls).

Hint: random.seed(42); [random.randint(1,100) for _ in range(5)]